```
# -*- coding: utf-8 -*-
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Imbalanced-Learn for SMOTE and Pipelines
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, StackingClassi
from xgboost import XGBClassifier

# Evaluation Metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score,
    precision_recall_curve, confusion_matrix, classification_report
    RocCurveDisplay, PrecisionRecallDisplay
)

RANDOM_STATE = 42
DATA_PATH = "core-temperature.csv"

# ==========================================
# 1. FEATURE ENGINEERING & PREPARATION
# ==========================================

def create_engineered_features(df):
    df = df.copy()
    temp_cols = ['TWATemp', 'LastReadingTemp', 'EndCaseTemp']
    if all(c in df.columns for c in temp_cols):
        # Calculate intraoperative temperature drops [cite: 111]
        df['TempDrop_TW_End'] = df['TWATemp'] - df['EndCaseTemp']
        df['TempDrop_Last_End'] = df['LastReadingTemp'] - df['EndCa
```

```
        # Calculate mean temperature profile [cite: 114]
        df['AvgTemp'] = df[temp_cols].mean(axis=1)
    return df

def prepare_data(df, use_engineered=True):
    df = df.copy()
    if use_engineered:
        df = create_engineered_features(df)
    target = 'AnyInfection'
    y = df[target]

    # Drop leakage columns as specified in the paper [cite: 86]
    drop_cols = ['AnyInfection','SeriousInfection','SuperficialInfe
                 'FascialDehiscence','DelayedHealing','Infection','
                 'Pneumonia(aspiration)','Sepsis','SSIDeep(fascia)'
                 'SSISuperficial(skin)','WoundInfection','LOS','Dur
    X = df.drop(columns=[c for c in drop_cols if c in df.columns])

    numeric = X.select_dtypes(include=['float64']).columns.tolist()
    categorical = X.select_dtypes(include=['object', 'category', 'b
    for col in X.select_dtypes(include=['int64']).columns:
        if col == 'YEAR' or X[col].nunique() <= 5: # Paper logic fo
            categorical.append(col)
        else:
            numeric.append(col)
    return X, y, numeric, categorical

# ==========================================
# 2. MODELING CONFIGURATION
# ==========================================

def get_base_models():
    """Optimized base models with cost-sensitive parameters[cite: 1
    return {
        'LogisticRegression': LogisticRegression(C=0.1, solver='lib
        'DecisionTree': DecisionTreeClassifier(max_depth=10, min_sa
        'RandomForest': RandomForestClassifier(n_estimators=100, ma
        'XGBoost': XGBClassifier(learning_rate=0.05, max_depth=3, s
    }

def tune_threshold_f2(y_true, y_proba):
    """Prioritizes Recall via F2-score tuning."""
    precision, recall, thresholds = precision_recall_curve(y_true,
    f2 = (5 * precision * recall) / (4 * precision + recall + 1e-9)
    return thresholds[np.argmax(f2)]

# ==========================================
# 3. VISUALIZATION ENGINE
# ------------------------------------------
```

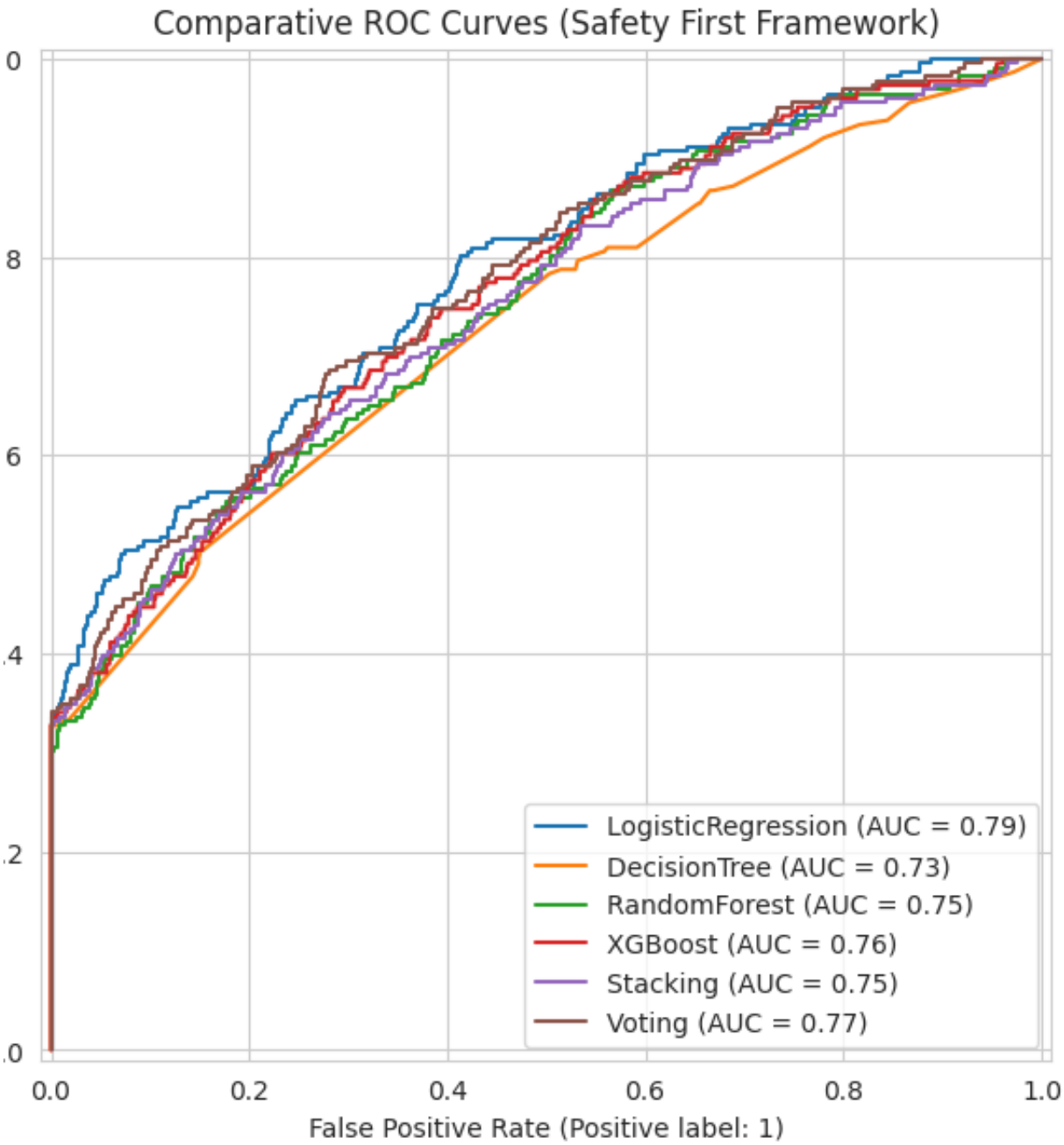
```
# ==========================================

def plot_paper_visuals(results, y_test):
    # Figure 1 & 2: ROC and PR Curves [cite: 226, 244]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 7))
    for name, data in results.items():
        RocCurveDisplay.from_predictions(y_test, data['proba'], nam
        PrecisionRecallDisplay.from_predictions(y_test, data['proba
    ax1.set_title("Comparative ROC Curves (Safety First Framework)"
    ax2.set_title("Comparative Precision-Recall Curves")
    plt.show()

    # Individual Confusion Matrices [cite: 290]
    num_models = len(results)
    cols = 3
    rows = (num_models // cols) + (1 if num_models % cols != 0 else
    fig, axes = plt.subplots(rows, cols, figsize=(18, 5 * rows))
    axes = axes.flatten()

    for i, (name, data) in enumerate(results.items()):
        cm = confusion_matrix(y_test, data['pred'])
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[
        axes[i].set_title(f"CM: {name}")
        axes[i].set_xticklabels(['No Inf (0)', 'Inf (1)'])
        axes[i].set_yticklabels(['No Inf (0)', 'Inf (1)'])
    plt.tight_layout()
    plt.show()

# ==========================================
# 4. EXECUTION LOOP
# ==========================================

def main():
    try:
        df = pd.read_csv(DATA_PATH)
    except:
        print("Data file not found.")
        return

    X, y, numeric, categorical = prepare_data(df)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_

    pre = ColumnTransformer([
        ('num', ImbPipeline([('imputer', SimpleImputer(strategy='me
        ('cat', ImbPipeline([('imputer', SimpleImputer(strategy='mo
    ])

    models = get_base_models()
```

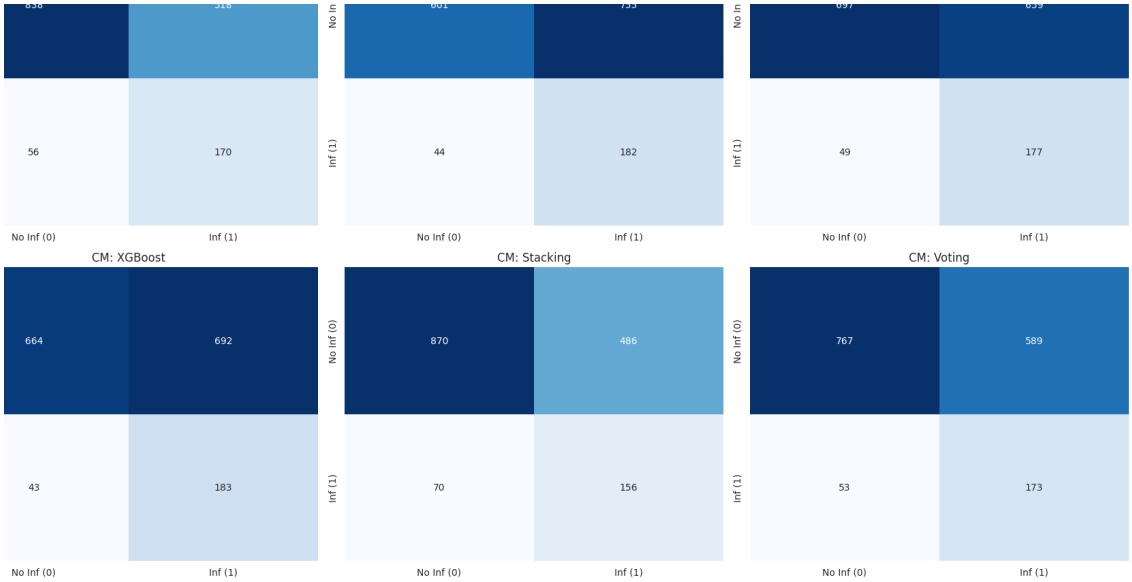
```
    # Build Ensemble Architecture [cite: 133, 142]
    estimators = [(n, m) for n, m in models.items()]
    models['Stacking'] = StackingClassifier(estimators=estimators,
    models['Voting'] = VotingClassifier(estimators=estimators, voti

    all_results = {}

    print("\n" + "="*60)
    print("CLINICAL RISK MODELING PERFORMANCE REPORT")
    print("="*60)

    for name, model in models.items():
        # Pipeline with SMOTE integration [cite: 122, 123]
        pipe = ImbPipeline([('pre', pre), ('smote', SMOTE(random_st
        pipe.fit(X_train, y_train)

        y_proba_train = pipe.predict_proba(X_train)[:, 1]
        y_proba_test = pipe.predict_proba(X_test)[:, 1]

        # Threshold optimized for Recall (Safety First) [cite: 26,
        thr = tune_threshold_f2(y_train, y_proba_train)
        y_pred = (y_proba_test >= thr).astype(int)

        all_results[name] = {'proba': y_proba_test, 'pred': y_pred}

        # DETAILED SCORES FOR 0 AND 1 [cite: 162, 168]
        print(f"\n--- MODEL: {name} ---")
        print(classification_report(y_test, y_pred, target_names=['
        print(f"ROC-AUC: {roc_auc_score(y_test, y_proba_test):.4f}"
        print(f"PR-AUC:  {average_precision_score(y_test, y_proba_t

    plot_paper_visuals(all_results, y_test)

if __name__ == "__main__":
    main()
```

```
============================================================
CAL RISK MODELING PERFORMANCE REPORT
============================================================

ODEL: LogisticRegression ---
              precision    recall  f1-score   support

fection (0)       0.94      0.62      0.74      1356
fection (1)       0.25      0.75      0.37       226

   accuracy                           0.64      1582
  macro avg       0.59      0.69      0.56      1582
```

```
eighted avg       0.84      0.64      0.69      1582

UC: 0.7856
C:  0.5820

ODEL: DecisionTree ---
              precision    recall  f1-score   support

fection (0)       0.93      0.44      0.60      1356
fection (1)       0.19      0.81      0.31       226

   accuracy                           0.49      1582
  macro avg       0.56      0.62      0.46      1582
eighted avg       0.83      0.49      0.56      1582

UC: 0.7325
C:  0.4882

ODEL: RandomForest ---
              precision    recall  f1-score   support

fection (0)       0.93      0.51      0.66      1356
fection (1)       0.21      0.78      0.33       226

   accuracy                           0.55      1582
  macro avg       0.57      0.65      0.50      1582
eighted avg       0.83      0.55      0.62      1582

UC: 0.7532
C:  0.5283

ODEL: XGBoost ---
              precision    recall  f1-score   support

fection (0)       0.94      0.49      0.64      1356
fection (1)       0.21      0.81      0.33       226

   accuracy                           0.54      1582
  macro avg       0.57      0.65      0.49      1582
eighted avg       0.83      0.54      0.60      1582

UC: 0.7638
C:  0.5429

ODEL: Stacking ---
              precision    recall  f1-score   support

fection (0)       0.93      0.64      0.76      1356
fection (1)       0.24      0.69      0.36       226

   accuracy                           0.65      1582
  macro avg       0.58      0.67      0.56      1582
eighted avg       0.83      0.65      0.70      1582
```

```
UC: 0.7529
C:  0.5391

ODEL: Voting ---
              precision    recall  f1-score   support

fection (0)       0.94      0.57      0.70      1356
fection (1)       0.23      0.77      0.35       226

   accuracy                           0.59      1582
  macro avg       0.58      0.67      0.53      1582
eighted avg       0.83      0.59      0.65      1582

UC: 0.7742
C:  0.5585
```




56
170
44
182
49
177
No Inf (0)
Inf (1)
CM: XGBoost
CM: Stacking
CM: Voting
664
692
870
486
767
589
43
183
70
156
53
173